第一节 格式化生成

从大语言模型（LLM）那里获得一段非结构化的文本在应用中常常不满足实际需求。为了实现更复杂的逻辑、与外部工具交互或以用户友好的方式展示数据，需要模型能够输出具有特定结构的数据，例如 JSON 或 XML。

本节将讨论实现格式化生成的几种主流方法，包括 LangChain、LlamaIndex 等框架内置的解决方案，不依赖框架的实现思路，以及一种更强大的技术——Function Calling。

> 在生成阶段，提示词工程也是一个重要的部分。但是因为在前面几个章节中已经有了比较多的介绍，所以本章就不再赘述了。

一、为什么需要格式化生成?

想象以下场景：

- **RAG 驱动的电商客服**：当用户询问“推荐几款适合程序员的键盘”时，我们希望 LLM 返回一个包含产品名称、价格、特性和购买链接的 JSON 列表，而不是一段描述性文字，以便前端直接渲染成商品卡片。
- **自然语言转 API 调用**：用户说“帮我查一下明天从上海到北京的航班”，系统需要将这句话解析成一个结构化的 API 请求，如 `{"departure": "上海", "destination": "北京", "date": "2025-07-18"}`。
- **数据自动提取**：从一篇新闻文章中，自动抽取出事件、时间、地点、涉及人物等关键信息，并以结构化形式存入数据库。

在这些场景中，格式化生成是连接 LLM 的自然语言理解能力和下游应用程序的程序化逻辑之间的关键。

二、格式化生成的实现方法

2.1 Output Parsers

LangChain 提供了一个强大的组件——`OutputParsers`（输出解析器），专门用于处理 LLM 的输出。它的核心思想是：

1. **提供格式指令**：在发送给 LLM 的提示（Prompt）中，自动注入一段关于如何格式化输出的指令。
2. **解析模型输出**：接收 LLM 返回的纯文本字符串，并将其解析成预期的结构化数据（如 Python 对象）。

LangChain 提供了多种开箱即用的解析器，例如：

- **StrOutputParser**：最基础的输出解析器，它简单地将 LLM 的输出作为字符串返回。
- **JsonOutputParser**：可以解析包含嵌套结构和列表的复杂 JSON 字符串。
- **PydanticOutputParser**：通过与 Pydantic 模型结合，可以实现对输出格式最严格的定义和验证。

接下来通过一个具体的代码示例，重点分析 `PydanticOutputParser` 的工作原理。它通过将用户定义的 Pydantic 数据模型转换为详细的格式指令，并注入到提示词中，来引导 LLM 生成严格符合该数据结构的 JSON 输出。最后再将模型返回的 JSON 字符串安全地解析为 Pydantic 对象实例。

```
# (此处省略了导入和 LLM 初始化代码)

# 1. 定义期望的数据结构
```

```python
class PersonInfo(BaseModel):
    """用于存储个人信息的数据结构。"""
    name: str = Field(description="人物姓名")
    age: int = Field(description="人物年龄")
    skills: List[str] = Field(description="技能列表")

# 2. 基于 Pydantic 模型，创建解析器
parser = PydanticOutputParser(pydantic_object=PersonInfo)

# 3. 创建提示模板，注入格式指令
prompt = PromptTemplate(
    template="请根据以下文本提取信息。\n{format_instructions}\n{text}\n",
    input_variables=["text"],
    partial_variables={"format_instructions": parser.get_format_instructions()},
)

# 4. 创建处理链 (假定 llm 已被初始化)
chain = prompt | llm | parser

# 5. 执行调用
text = "张三今年30岁，他擅长Python和Go语言。"
result = chain.invoke({"text": text})

# 6. 打印结果
print(result)
# name='张三' age=30 skills=['Python', 'Go语言']
```

1. **定义数据模型**：使用 Pydantic 的 `BaseModel` 定义 `PersonInfo` 类，这不仅是一个 Python 对象，更是一个清晰的数据结构规范（Schema）。`Field` 中的 `description` 描述文本将直接作为指令提供给大模型，因此其表述需要清晰准确。
2. **生成格式指令**：当 `PydanticOutputParser` 实例化后，其 `get_format_instructions()` 方法会执行以下操作：
    - 调用 Pydantic 模型的 `.model_json_schema()` 方法，提取出该数据结构的 JSON Schema 定义。
    - 对该 Schema 进行简化，并将其嵌入到一个预设的、指导性的提示模板中。这个模板明确要求 LLM 输出一个符合该 Schema 的 JSON 对象。
3. **构建并执行调用链**：通过 LangChain 表达式语言（LCEL），将 `prompt`、`llm` 和 `parser` 链接起来。当调用链被触发时：
    - `prompt` 会将用户输入（`text`）和上一步生成的格式指令（`format_instructions`）组合成最终的提示，发送给 `llm`。
    - `llm` 根据这个包含严格格式要求的提示，生成一个 JSON 格式的字符串。
4. **解析与验证**：`PydanticOutputParser` 接收到 LLM 返回的字符串后，会执行一个两步解析过程：
    - 首先，它继承自 `JsonOutputParser`，会将 LLM 输出的文本字符串解析成一个 Python 字典。
    - 然后，最关键的一步，它会使用 `PersonInfo.model_validate()` 方法，用定义的数据模型来验证这个字典。如果字典的键和值类型都符合 `PersonInfo` 的定义，解析器就会返回一个 `PersonInfo` 的实例对象；如果验证失败，则会抛出一个 `OutputParserException` 异常。

> [完整代码]

2.2 LlamaIndex 的输出解析

LlamaIndex 的输出解析与生成过程紧密结合，主要体现在两大核心组件中：响应合成（Response Synthesis）和结构化输出（Structured Output）。

1. **响应合成**

   在 RAG 流程中，检索器召回一系列相关的文本块（Nodes）后，并不是简单地将它们拼接起来。响应合成器（Response Synthesizer）负责接收这些文本块和原始查询，并以一种更智能的方式将它们呈现给 LLM 以生成最终答案。例如，它可以逐块处理信息并迭代地优化答案（`refine` 模式），或者将尽可能多的文本块压缩进单次 LLM 调用中（`compact` 模式）。这个阶段的默认目标是生成一段高质量的**文本**回答。

2. **结构化输出**

   当需要 LLM 返回结构化数据（如 JSON）而非纯文本时，LlamaIndex 主要使用 **Pydantic 程序（Pydantic Programs）**。这与 LangChain 的 `PydanticOutputParser` 思想一致：

   - **定义 Schema**：开发者首先定义一个 Pydantic 模型，明确所需输出的数据结构、字段和类型。
   - **引导生成**：LlamaIndex 会将这个 Pydantic 模型转换成 LLM 能理解的格式指令。如果底层的 LLM 支持 Function Calling，LlamaIndex 会优先使用该功能以获得更可靠的结构化输出。如果不支持，它会回退到将 JSON Schema 注入到提示词中的方法。
   - **解析验证**：最后，LLM 返回的输出会被自动解析并用 Pydantic 模型进行验证，确保其类型和结构完全正确，最终返回一个 Pydantic 对象实例。

2.3 不依赖框架的简单实现思路

如果你不想依赖特定的框架，也可以通过提示工程（Prompt Engineering）的技巧来实现格式化生成。

主要思路是在提示中给出清晰、明确的指令和示例。以下是一些实用技巧：

- **明确要求 JSON 格式**：在提示中直接、强硬地要求模型“必须返回一个 JSON 对象”、“不要包含任何解释性文字，只返回 JSON”。
- **提供 JSON Schema**：在提示中给出你想要的 JSON 对象的模式（Schema），描述每个键的含义和数据类型。
- **提供 few-shot 示例**：给出 1-2 个“用户输入 -> 期望的 JSON 输出”的完整示例，让模型学习输出的格式和风格。
- **使用语法约束**：对于一些本地部署的开源模型（如通过 `llama.cpp` 运行的模型），可以使用 GBNF (GGML BNF) 等语法文件来强制约束模型的输出，确保其生成的每一个 token 都严格符合预定义的 JSON 语法。这是最严格也是最可靠的非 Function Calling 方法。

三、Function Calling

Function Calling（或称 Tool Calling）是近年来 LLM 领域的一个重要进展，提升了模型与外部世界交互和生成结构化数据的能力。

3.1 概念与工作流程

Function Calling 的本质是一个多轮对话流程，让模型、代码和外部工具（如 API）协同工作。其核心工作流如下：

1. **定义工具**：首先，在代码中以特定格式（通常是 JSON Schema）定义好可用的工具，包括工具的名称、功能描述、以及需要的参数。
2. **用户提问**：用户发起一个需要调用工具才能回答的请求。

3. **模型决策**：模型接收到请求后，分析用户的意图，并匹配最合适的工具。它不会直接回答，而是返回一个包含 `tool_calls` 的特殊响应。这个响应相当于一个指令："请调用某某工具，并使用这些参数"。
4. **代码执行**：应用接收到这个指令，解析出工具名称和参数，然后**在代码层面实际执行**这个工具（例如，调用一个真实的天气 API）。
5. **结果反馈**：将工具的执行结果（例如，从 API 获取的真实天气数据）包装成一个 `role` 为 `tool` 的消息，再次发送给模型。
6. **最终生成**：模型接收到工具的执行结果后，结合原始问题和工具返回的信息，生成最终的、自然的语言回答。

3.2 Function Calling 实践

接下来，直接使用 `openai` 的例子，来展示上述流程。

```
# 1. 定义工具
tools = [...]

# 2. 用户提问
messages = [{"role": "user", "content": "杭州今天天气怎么样？"}]
message = send_messages(messages, tools=tools)

# 3. 代码执行：模拟调用天气API，并将结果添加到消息历史
if message.tool_calls:
    tool_call = message.tool_calls[0]
    messages.append(message) # 添加模型的回复
    tool_output = "24℃，晴朗" # 模拟API结果
    messages.append({
        "role": "tool",
        "tool_call_id": tool_call.id,
        "content": tool_output
    }) # 添加工具执行结果

    # 4. 第二次调用 (`Tool -> Model`)：将工具结果返回给模型，获取最终回答
    final_message = send_messages(messages, tools=tools)
    print(final_message.content)
```

关键步骤：

1. **定义 `tools`**：用一个列表包含了所有可用的函数定义。每个定义都是一个 JSON 对象，严格描述了函数的名称（`name`）、功能（`description`）和参数（`parameters`）。这个描述的质量直接决定了模型能否正确选择和使用工具。
2. **第一次调用（`User -> Model`）**：将用户的原始问题（`"role": "user"`）和 `tools` 列表一同发送给模型。
3. **处理 `tool_calls`**：检查模型的响应中是否包含 `tool_calls`。如果包含，就说明模型决定使用工具。解析出函数名和参数，并**模拟执行**（在真实场景中，这里会是真实的 API 调用）。
4. **第二次调用（`Tool -> Model`）**：将原始的用户问题、模型的工具调用响应，以及模拟执行后得到的工具结果（`"role": "tool"`），一同打包成新的对话历史，再次发送给模型。
5. **获取最终答案**：模型在看到工具的执行结果后，就能用自然语言回答用户最初的问题了。

> 完整代码

3.3 Function Calling 的优势

相比于单纯通过提示工程“请求”模型输出 JSON，Function Calling 的优势在于：

- **可靠性更高**：这是模型原生支持的能力，相比于解析可能格式不稳定的纯文本输出，这种方式得到的结构化数据更稳定、更精确。
- **意图识别**：它不仅仅是格式化输出，更包含了“意图到函数的映射”。模型能根据用户问题主动选择最合适的工具。
- **与外部世界交互**：它是构建能执行实际任务的 AI 代理（Agent）的核心基础，让 LLM 可以查询数据库、调用 API、控制智能家居等。